



SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Adaptation Techniques for using NAS
Methods in the FL Setting**

Max Coetzee





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

**Adaptation Techniques for using NAS
Methods in the FL Setting**

Titel

Author:	Max Coetzee
Examiner:	Supervisor
Supervisor:	Advisor
Submission Date:	Submission date



I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, Submission date

Max Coetzee

Acknowledgments

Abstract

Both *Neural Architecture Search* (NAS) and *Federated Learning* (FL) have made significant progress independently in the past decade. To benefit from the advantages of NAS methods in FL, researchers have started combining them by using NAS in FL [9] [4] [21].

[TODO: overhaul]

Applying techniques from Neural Architecture Search (NAS) to Federated Learning (FL) has been fruitful (remove) in recent years. The combination was identified as a promising research (remove) direction by [15]. It has yielded methods for finding architectures that deal with the challenges imposed by the FL setting.

Research into NAS has grown rapidly [37] since it was popularized by [54]; consequently, literature on its application to FL has grown. The last survey on NAS applied to FL compared approaches of four papers [53]. Since then, we have identified approximately 50 new papers. This motivates a new systematic survey of the landscape to identify progress and gaps in the literature.

In this thesis, we propose a map of the literature landscape based on the FL challenges they address. We achieve this by systematically evaluating the literature and identifying which challenge it solves.

We refer to the FL challenges described in [54], i.e., non-IID data, limited communication, client heterogeneity, privacy of client data, and break them down into smaller subchallenges — each subchallenge being associated with a pattern in the literature. We include personalized FL [34] as an additional subchallenge that was not originally posited, but has since drawn the community’s attention.

We then analyze how the subchallenges are addressed and focus on the contribution of the used NAS method towards overcoming the subchallenge. For each subchallenge, we keep track of the NAS types used (following [37], [1]) and assess whether the underexplored methods are candidates for future research.

Neural Architecture Search

Contents

Acknowledgments	iv
Abstract	v
1 Introduction	1
1.1 Background	3
1.1.1 Neural Architecture Search (NAS)	3
1.1.2 Federated Learning (FL)	4
1.1.3 NAS in FL	4
2 Method	6
2.1 Literature Selection	6
2.2 Literature Analysis	6
2.3 Reviewed Literature	8
3 Challenges and Solutions for Using NAS in FL	9
3.1 Resource Burden of NAS in FL	9
3.1.1 Computation and Search Effort	9
3.1.2 Communication Burden	13
3.1.3 Energy Burden	15
3.2 Client Heterogeneity	15
3.2.1 Hardware Heterogeneity	15
3.2.2 Client Data Heterogeneity	17
3.3 Federated Search-State Coordination	19
3.3.1 Coordinating Distributed Search State	19
3.3.2 Coordinating Interdependent Search Decisions	21
3.3.3 Coordinating Vertically Partitioned Search	21
3.4 Disrupted Architecture Search	22
3.4.1 Participation Disruption and Bias	22
3.4.2 Structural Instability	23
3.5 Architecture Search Security	24
4 Conclusion	27

Contents

5 Discussion	28
5.1 Principal Findings	28
5.2 Implications for Practice and Research	30
5.3 Limitations	31
5.4 Future Research	32
Abbreviations	33
List of Figures	34
List of Tables	35
Bibliography	36

1 Introduction

Deep learning has seen widespread adoption over the last decade and is being applied to an increasingly diverse set of tasks. Applying deep learning traditionally involves a team of deep learning and domain experts who tailor a neural network architecture to a specific task through a lengthy process of trial and error. To automate this process, researchers invented *Neural Architecture Search* (NAS) [6] methods. NAS methods employ diverse strategies to automatically search for a neural network architecture for a given deep learning task within an architecture search space.

Independently, but in parallel to NAS, researchers developed a machine learning approach called *Federated Learning* (FL) [25], which enables training an ML model on data distributed across a set of clients without sharing their data. The model is distributed from the server to the clients, which train the model on their local data. The clients then send the model weight updates to the server, which aggregates them and redistributes them to the clients. This process is repeated for several rounds. FL enhances the privacy of client data and enables training even when clients' data is too large to transfer effectively to a central training site or cannot be shared due to regulatory or privacy constraints.

There has been significant interest in using NAS in FL over the last couple of years, but using NAS methods in FL is not straightforward. Most NAS methods were developed for a centralised setting and rely on assumptions that often do not hold for FL. These assumption discrepancies create several *challenges* for using NAS in FL. For example, in a centralised setting, practitioners can expect a NAS method to run on worker nodes connected via low-latency, high-bandwidth links. However, in FL, a NAS method may run on clients connected via an unreliable, high-latency, low-bandwidth network. This creates a challenge when transferring the weights of large candidate models between clients and the server to coordinate an architecture search. Practitioners need specialised solutions to overcome such challenges, such as only transferring encodings of candidate architectures to clients [32].

The NAS-in-FL literature contains many NAS-in-FL methods that propose various solutions for overcoming the challenges of using NAS in FL [10]. As NAS-in-FL research has progressed, several overarching challenges and solution themes have appeared in the literature. Despite differences in implementation and terminology, solutions are often conceptually similar. For example, different methods use partial-parameter

aggregation or knowledge transfer to coordinate searches involving heterogeneous architectures [rafl_2024, 4, 41].

Earlier reviews classify methods as online or offline and single- or multi-objective, summarise a small set of NAS-in-FL methods within a broader NAS and hyperparameter optimisation review, or discuss NAS as one part of multi-objective FL [53, 17, 8]. More recent work supports comparison by conceptualising a generic NAS-in-FL pipeline, design elements, and method archetypes [10]. While these reviews provide an overview of the types of NAS-in-FL methods, they do not highlight individual challenges of NAS-in-FL methods and the solutions covered in the literature to overcome them. NAS-in-FL research lacks a concept-centric overview of its problem and solution space. Individual studies describe challenges and solutions for particular NAS methods and FL settings using different terminology and levels of abstraction, but they do not reveal which challenges and solutions recur across the literature as overarching concepts.

While themes in challenges and solutions have emerged, they can not be applied universally, since they depend on the FL system characteristics and the used NAS-in-FL method. Furthermore, seemingly related challenges are quite different upon closer inspection and warrant different solutions. A clear overview of the challenge and solution space for NAS-in-FL is needed to make practitioners aware under which circumstance which potential pitfalls occur and make them aware for which circumstances solutions already exist and where work can be done. This leads us to ask the following research question:

Which <adjective placeholder> solutions does the literature propose for addressing the challenges that arise when using NAS in FL?

To tackle our research question, we conduct an extensive systematic literature review. We collect an initial set of relevant literature with a Scopus [5] search and extend it with a snowball search. We first use open coding to identify, from the literature, assumption discrepancies and resulting challenges, when using NAS in FL. Next, we extract FL system characteristics and use axial coding to code the influence of characteristics on the relevance of challenges. Then, we extract techniques through open coding and iteratively refine them to synthesise a coherent set of mutually exclusive and collectively exhaustive techniques. Finally, we use axial coding to map techniques onto challenges they overcome.

Our review aims to synthesise techniques for using NAS in FL from the literature to make finding suitable techniques easy in practice. We make the following three contributions. First, by providing a consolidated overview of the challenges of using NAS in FL, practitioners can grasp which issues they can expect to encounter at a glance. Second, by describing how FL system characteristics influence the relevance of challenges, we enable practitioners to focus on the challenges relevant to them. Third,

by extracting techniques for using NAS in FL and highlighting the challenges they address, we enable practitioners to compare existing techniques for using NAS in FL for their specific set of challenges.

In Chapter ??, we cover the background required for this thesis and related work. In Chapter 2, we describe the method with which we perform our literature review and give an overview of the included literature. In Chapter ??, we highlight the challenges with FedNAS and how they arise from different FL settings. In Chapter ??, we describe the adaptation techniques we synthesised and how they overcome challenges. In Chapter 5, we conduct a discussion about our work. Chapter 4 contains our conclusion.

1.1 Background

1.1.1 Neural Architecture Search (NAS)

Neural architecture search (NAS) automates the design of a neural network architecture. There are many different ways to perform NAS, resulting in different *NAS methods*. A NAS method consists of a search space, a search strategy, and a performance-estimation strategy [6, 37].

The search space defines the potential candidate architectures. Search spaces can be cell-based, ... or

including their operations, connections, depth, and width. The search strategy uses observations gathered during the search to select further candidates, while performance estimation approximates how candidates would perform after full training. A search can optimise predictive performance alone or add objectives such as model size, latency, energy consumption, and communication cost.

Search strategies differ in their required work and state. Black-box strategies select discrete candidates and use measured or predicted performance to guide subsequent selections [37].

One-shot strategies represent many candidates within a shared supernet, from which subnetworks inherit weights. Differentiable variants maintain continuous architecture parameters, whereas other variants sample or progressively prune subnetworks. Weight sharing reduces search cost but introduces a large shared state and can make inherited-weight performance an unreliable proxy for independently trained performance [37].

Performance estimation further changes the cost of either strategy. Full training provides high-fidelity evidence at substantial cost; shortened training and predictors trade estimation fidelity for speed [37]. Together, the three NAS components determine which computations clients perform, what search state they store and exchange, and how frequently they evaluate architectures.

1.1.2 Federated Learning (FL)

Federated learning (FL) enables clients to train a model collaboratively while keeping their raw data local. In a common server-coordinated process, the server selects clients and distributes the global model. Each selected client trains the model locally and returns an update, which the server aggregates for the next communication round [25]. The server consequently receives data-dependent signals; additional privacy mechanisms remain necessary when those signals are sensitive [15].

FL systems vary in scale and client characteristics. Cross-device FL commonly involves many intermittently available mobile or Internet of Things devices, of which only a subset participates in each round. These clients have limited and heterogeneous computation, memory, energy, and wide-area bandwidth [15].

Cross-silo FL usually involves fewer, better-resourced organisations or data centres with stable identities and reliable participation [15]. Its data can be partitioned horizontally, with clients holding different samples, or vertically, with parties holding different features for overlapping samples. In both settings, data quantities and distributions can differ between clients. These settings describe characteristic configurations rather than an exhaustive dichotomy, but establish different expectations about resource constraints, reliability, coordination, and trust.

1.1.3 NAS in FL

NAS in FL incorporates architecture search into the federated process without centralising client data. Conventional NAS training becomes local client training, while architecture instantiation, evaluation, search-state updates, and selection can occur at the server, at clients, or at both [10]. Clients may exchange model weights together with architecture parameters, candidate encodings, controller state, performance scores, or intermediate representations. The search can produce one global model or client- and group-specific models. Candidate quality also depends on its federated training configuration, coupling architecture search with choices such as local epochs and client participation [10].

The search strategy determines how strongly this additional work affects an FL system. Full-supernet gradient and pruning searches impose high per-round computation and memory demands and require a common supernet representation. Subnet-gradient searches reduce individual client work but must reconcile updates to different parts of the supernet. Population- and controller-based searches place smaller candidates on clients and support parallel evaluation, although repeated evaluations can increase total computation and communication [10]. Surrogate evaluation and weight inheritance reduce this cost while making candidate rankings depend on proxy evidence [37, 10].

Challenge applicability therefore follows from the combination of FL system and NAS characteristics. Limited resources, dropout, and unequal search participation are especially severe when a one-shot search runs across many cross-device clients. A cross-silo system may make supernet training feasible, while vertically partitioned data introduces end-to-end coordination and intermediate-representation exchanges [15, 10]. Security concerns apply across settings, although the exposed search signal and the parties able to observe it depend on the chosen search.

Solutions inherit the same context dependence: a mechanism that reduces the size of discrete candidate messages does not address a full-supernet memory limit, while aggregation of architecture parameters presumes a compatible shared representation. The challenge–solution analysis therefore considers both the FL system characteristics and the NAS design to delimit where each challenge and solution applies.

2 Method

2.1 Literature Selection

In accordance with Webster and Watson, we did...

TODO: include prisma chart (applicable with webster and watson?)

2.2 Literature Analysis

inductive thematic analysis

We conduct a concept-centric literature review following [webster_watson_literature_review_2002] and analyse the included publications using thematic analysis [using_thematic_analysis_2006].

As the starting point for our synthesis, we construct the concept matrix proposed by [webster_watson_literature_review_2002]. For each publication, we use the matrix to record passages describing challenges caused by using NAS in FL, proposed solutions and the relevant FL system characteristics. The matrix preserves the link between publications and concepts while organising the synthesis around recurring challenges and solutions instead of individual NAS-in-FL methods.

We apply the six stages of thematic analysis through three analytical iterations inspired by [smart_contracts_challenges_solutions_2022] and [14]. In the first iteration, we familiarise ourselves with the complete literature set and populate the concept matrix with initial semantic codes. Each code records the meaning of a passage describing an FL system characteristic, a challenge or a solution without assigning it to a predetermined theme.

In the second iteration, we search for themes by comparing codes across publications and collating conceptually equivalent descriptions. Related codes form recurring challenges and solution types, which we arrange into candidate subthemes and overarching themes. A theme captures a repeated pattern relevant to the research question, while a subtheme distinguishes a coherent pattern within a broader theme. We retain patterns for which the literature provides sufficient evidence to define their meaning and applicability.

In the third iteration, we review the candidate themes against both their coded passages and the complete literature set. We revise the FL system characteristics,

reassign codes and merge or split themes until the contents of each theme cohere and the distinctions between themes are clear. We then define and name each theme through its central concept and produce the report by relating its concrete challenges and recurring solutions to the applicable FL system characteristics and NAS approaches.

[TODO: overhaul to include method for identifying challenges and FL settings] - targeted FL setting can be stated directly, indirectly or must be inferred from experimental setup

We use various techniques from [3] and lean on parts of the methodology of [14] to perform a systematic literature review.

1. **Literature Selection:** We follow the guidelines and flow diagrams provided by PRISMA 2020 [28] for inclusion and exclusion of papers and perform forward and backwards citation searching. We identify 59 papers that present FedNAS methods. Each paper contains one or more FedNAS methods.
2. **Unrefined Adaptation Technique Extraction:** Once the set of included papers is fixed, we perform open coding on each FedNAS method to extract unrefined adaptation techniques. Any modification to a NAS method that is explicitly motivated by the federated setting is treated coded as one unrefined adaptation technique.
3. **Adaptation Techniques Conceptualization:** We iteratively refine and merge unrefined adaptation techniques in an axial coding step to obtain a coherent set of adaptation techniques that are mutually exclusive and collectively exhaustive. Unrefined adaptation techniques with conceptually highly-similar mechanisms are merged into a single representative adaptation technique.
4. **Categorise Adaptation Techniques:** After merging, in a second axial coding step, we cluster adaptation techniques based on a) the FedNAS challenges they address and b) the conceptual similarity of their mechanisms. As a result, we obtain a taxonomy of adaptation techniques.
5. **Discuss FedNAS Challenges for Adaptation Techniques:** We discuss how each adaptation technique works towards, against, or does not affect overcoming each of FedNAS challenges.
6. **Produce Table Overviews:** Finally, we create two tables that practitioners can use to make decisions about FedNAS methods for their use case.

The first table contains a coded vector of effects over the FedNAS challenges for each *FedNAS method*. The effects per FedNAS method are the result of the effects of its adaptation techniques in aggregate. Practitioners can use this table

to decide which FedNAS method suits their use case or if their use case would benefit from a new FedNAS method.

The second table contains a coded vector of effects over the FedNAS challenges for each *adaptation technique* based on the prior discussion. If practitioners need to create new FedNAS methods, they can use this table to choose existing adaptation techniques relevant to the set of FedNAS challenges they need to address for their use case.

2.3 Reviewed Literature

add some overview and statistics about the literature

3 Challenges and Solutions for Using NAS in FL

Our thematic analysis revealed five overarching themes for challenges: *Resource Burden of NAS in FL*, *Client Heterogeneity*, *Federated Search-State Coordination*, *Disrupted Architecture Search* and *Architecture Search Security*. Each solution remains with the primary challenge that it addresses, while the subsections identify recurring patterns within the broader themes.

- FL system characteristics for which the challenges are applicable/more severe - NAS types based on NAS in FL and NAS 1000 papers categorisation

3.1 Resource Burden of NAS in FL

NAS may train or estimate the performance of many candidates, while distributing this work across clients adds resource costs to an already expensive procedure [ying2019nasbench101reproduci15]. This theme covers challenges that make federated architecture search operationally burdensome and solutions that reduce, reuse or reorganise the required work.

3.1.1 Computation and Search Effort

Challenge 1.1: Evaluating Candidate Architectures on Constrained Computational Resources

NAS-in-FL can become infeasible when the model that a client must evaluate in one communication round exceeds its available memory capacity or the evaluation cannot be completed within the time available for a communication round. This concern is central to approaches that target mobile and edge clients [4, 46, 7] where client memory and computational power is rather limited. The problem is particularly acute for Differentiable Supernet-Based strategies that distribute a shared supernet and require clients to update multiple mixed candidate operations locally [dp-fnas_2020, 9, 30, 38]. Other search strategy types usually assign one discrete or generated candidate at a time, but it remains challenging to evaluate even just one candidate model in a feasible amount of time.

Solution 1.1a: Build the search space from inexpensive components. Practitioners can apply this solution to every search strategy type by restricting candidate architectures to inexpensive operations, blocks or modules, depending on the search space type. Prominent examples include depthwise-separable convolutions and mobile-oriented CNN blocks [38, 31]. This makes client-side evaluation cheaper, but may exclude expensive components that increase prediction accuracy.

Solution 1.1b: Train only a subnet per client at a time. Supernet-based search strategies can be modified so that clients train and evaluate a subnet corresponding to a subset of candidate architectures instead of a weighted mixture of all candidate operations, reducing the computational burden on individual clients [7]. The subnet size can be adjusted dynamically to the computational budget of each client, with the most extreme case being evaluation of a single path [**divide-and-conquer_fednas_2023**, 4, 48, 36, 20]. However, sampling only part of the supernet in each round can evaluate candidate operations unevenly and thereby bias comparisons between candidate architectures [4, 41].

Solution 1.1c: Search trainable adaptation modules over a frozen backbone. When a suitable pretrained model is available, clients can keep its backbone fixed and search a sparse combination of lightweight adaptation modules. Only the selected modules and their architecture variables require local optimisation and exchange, reducing both client computation and update size [**fednaspet_2023**]. The benefit depends on how well the fixed backbone transfers to the clients' tasks.

Challenge 1.2: Reducing the Cost of the Federated Search Across Rounds

Even when evaluating candidates on individual clients is feasible, keeping the overall duration of the federated architecture search within a practical time frame can be difficult because the search must compare architectural alternatives across repeated rounds in addition to optimising model weights. Black-box strategies may train several discrete candidates through separate federated evaluations [45, 52], while weight-sharing strategies repeatedly update an over-parameterised supernet or its sampled subnets [9, 18]. A separate post-search training cost applies only when the selected architecture cannot reuse search-time weights, as discussed in Challenge 1.3.

Solution 1.2a: Dynamic candidate evaluation budgets. This solution applies to black-box search strategies that compare discrete candidates through independent training, including greedy pruning, evolutionary and genetic algorithms, reinforcement learning and Bayesian optimisation. Similar to multi-armed-bandit AutoML approaches [**automl_book_hpo_2019**], each candidate initially receives only a small budget of federated rounds, after which weak candidates can be discarded and promising candidates trained for longer.

For example, [45, 46] observes that the best candidate usually becomes distinguishable within the first couple of communication rounds, drops the remaining candidates and increases the round budget in later search iterations. Evolutionary searches similarly use a fixed small number of rounds or early stopping to limit candidate evaluation [gpt4-boosted_pso_fednas_2024, 51].

Solution 1.2b: Prune the supernet progressively. Supernet-based strategies can periodically remove weak candidate operations from the shared supernet, so fewer alternatives remain stored and trained in subsequent federated rounds [7, 40]. Unlike dynamic candidate evaluation budgets, this changes the search space itself rather than the number of rounds allocated to independently trained candidates. Its savings increase as the search progresses, but the initial rounds still incur the cost of the complete supernet and early pruning can permanently remove useful operations.

Solution 1.2c: Predict candidate performance. Performance prediction is a standard AutoML technique for reducing the number of costly configuration evaluations [automl_book_hpo_2019]. In NAS-in-FL, candidate-based strategies can evaluate a small set of architectures and train a predictor to estimate the quality of the remaining candidates [41, 33]. Additional care is required because private and non-IID client data can make architecture rankings client-specific. Predictors can therefore be trained separately for each client [41], or learned collaboratively by aggregating their parameters without transmitting the locally evaluated architectures [22]. This reduces the number of candidates evaluated by clients, but its reliability depends on whether the evaluated architectures represent the relevant parts of the search space.

Solution 1.2d: Evaluate candidates in parallel. Since black-box search strategies generate several distinct candidates per iteration, the candidates can be trained and evaluated concurrently by separate client groups [45, 52, 33].

Supernet-based strategies can instead sample several subnets, allocate each to a client cluster and merge their updates back into the shared supernet [21]. This potentially reduces elapsed search time when enough clients are available, but comparisons can become biased when the groups evaluate candidates on different data distributions [45].

Solution 1.2e: Reuse supernet weights for candidate evaluation. Weight-sharing strategies can train common operation weights and allow each candidate to inherit the subset associated with its architecture [35, 48]. This avoids training every candidate independently from scratch, although inherited-weight performance can rank candidates differently from full training. A short incremental federated fine-tuning stage can provide a higher-fidelity estimate for a smaller set of promising candidates [39].

Solution 1.2f: Supervise concurrent search agents. Several search agents can run in parallel while a supervisor compares their relative losses and adjusts the momentum applied by each agent. The final architecture can then be selected according to prediction error, training time and model size [supervised_fednas_2023]. This can smooth and

accelerate convergence when parallel resources are available, but requires maintaining several search trajectories.

Challenge 1.3: Post-Search Retraining Duplicates Federated Computation

Some NAS strategies use search-time training only to select an architecture and then train the selected model from a new initialisation. In NAS-in-FL, this adds another sequence of client-side epochs and aggregation rounds after clients have already spent computation and communication on the search [39, 49]. This challenge applies whenever the search produces an architecture without reusable final weights and therefore delays the availability of a deployable model.

Solution 1.3a: Reuse weights learned during search. Weight-sharing supernet strategies can initialise the selected subnet with the corresponding operation weights learned during search and perform only limited final fine-tuning [39]. This reuses client computation that has already been spent on the selected operations, although weights learned jointly with competing subnets may be a poor initialisation for the final model.

Solution 1.3b: Produce a deployable model during search. One-stage supernet-based strategies can jointly optimise architecture selection and model weights while concentrating the architecture distribution or pruning weak operations, so the search terminates with a discrete model and trained weights [7, 12]. Reinforcement-learning strategies over a weight-sharing supernet can similarly turn later search rounds into fine-tuning as their sampling probabilities concentrate [42]. This eliminates the separate retraining stage, but the final path may remain undertrained if it received too few updates while sharing weights with competing paths.

Challenge 1.4: Separate Searches for Deployment Targets Multiply Cost

A NAS-in-FL deployment may require different architectures for several latency or computation budgets. Searching and training independently for each target repeats the federated procedure, so the required client computation and communication grow with the number of device classes [18].

Solution 1.4a: Reuse one supernet across deployment targets. A weight-sharing supernet containing subnets of different depths and widths can be trained once through FL. Clients can then use predictor-guided local search to select subnets for their deployment constraints without further federated training. This amortises the training cost across targets and has reduced it elevenfold for 20 targets [18]. Co-training many subnets can, however, cause weight-sharing interference and leave rarely sampled parameters stale [18].

Challenge 1.5: Split-Model Search Leaves Client Computation Idle

In split-model training, different clients or device groups execute consecutive model segments and exchange intermediate activations and gradients. The resulting sequential dependency leaves clients waiting while other segments execute. NAS-in-FL methods that distribute a search model in this manner inherit these idle periods during architecture search [13].

Solution 1.5a: Search locally during split-training idle periods. Auxiliary input and output components can turn an assigned model segment into a locally executable supernet. During otherwise idle periods, the client samples and trains alternative branches, using the corresponding segment of the split model as a teacher for feature distillation [13]. This converts waiting time into architecture-search work that can proceed without the remaining model partitions.

3.1.2 Communication Burden

Cross-device clients often communicate with the server over capacity-limited wide-area links. Typical wide-area bandwidths of 5–25 Mbit/s are far below the more than 10 Gbit/s available within data centres [40]. The discussion below assumes a common bandwidth limit. Unequal connectivity is treated separately as client heterogeneity.

Challenge 1.6: Search Messages Exceed Client Transfer Budgets

Fixed-model FL exchanges the parameters of one architecture. NAS-in-FL may additionally exchange an over-parameterised supernet or architecture variables. A supernet contains weights for mutually exclusive operations and can therefore be much larger than any selected architecture, making the complete search state infeasible to transfer over a constrained client link [4, 41].

Solution 1.6a: Compress search updates. Quantisation represents update entries with fewer bits, while sparsification transmits only selected entries. One differentiable supernet-based approach uses 8-bit stochastic quantisation and retains the largest 20% of entries after clipping [2]. These operations reduce the transferred weight and architecture updates without changing the search variables.

Solution 1.6b: Encode architecture choices compactly. When clients already hold a compatible master model or trained supernet, the server can identify a candidate using a selector or genotype instead of transmitting its parameters. Clients reconstruct the indicated subnet locally and return its evaluation or updated weights [52, 39, 31]. This requires the server and clients to share the search-space definition and clients to retain the common weights.

Challenge 1.7: Architecture Search Adds Repeated Synchronisation

Differentiable NAS-in-FL updates architecture variables alongside model weights. Synchronising both in every round adds a communication channel that fixed-model FL does not require [9, 2]. Even small architecture updates can consume substantial bandwidth when exchanged throughout a long search.

Solution 1.7a: Synchronise architecture variables less frequently. Architecture variables can be treated as slow variables that are updated and uploaded less frequently than model weights. One approach transmits them every H_α rounds and stops doing so after the architecture has been fixed, while ordinary weight training continues [2].

Solution 1.7b: Aggregate search updates hierarchically. A nearby aggregator can combine the model and architecture updates of several clients before forwarding one result to the remote server. Frequent aggregation then uses local links, while fewer exchanges traverse the wide-area network [21]. In one non-IID CIFAR-10 evaluation, this reduced traffic at 80% target accuracy from 3.29–4.68 GB to 1.64 GB [21].

Solution 1.7c: Parallelise architecture and weight optimisation. Vertically partitioned differentiable strategies can formulate the architecture and weight updates independently within a round, allowing both to proceed in parallel rather than through alternating steps. This removes a sequential dependency and reduces the synchronised exchanges needed to update both variable sets [23].

Challenge 1.8: Communication-Unaware Search Produces Expensive Federated Models

An architecture selected only for predictive performance or local computation can still contain many parameters that FL must transmit in every subsequent training round. The search decision therefore determines the continuing load on client links, and a model that meets computation and memory budgets may still exceed the transfer budget [51].

Solution 1.8a: Optimise for transmitted model size. Communication cost can be included directly in the architecture objective. Evolutionary search can optimise global error together with the average number of weights uploaded by clients, favouring sparse architectures [51]. Differentiable supernet-based search can similarly regularise the expected number of transmitted bytes [2]. Both expose the trade-off between predictive performance and continuing communication cost before selecting an architecture.

3.1.3 Energy Burden

Challenge 1.9: Search Computation and Communication Deplete Client Energy

Message size and round duration do not fully represent the burden on battery-powered clients. Repeated local optimisation consumes computation energy, while every search-state exchange consumes transmission energy. Fixed local epoch counts can create redundant communication in some rounds and unnecessary computation in others, increasing the energy required to reach a target performance [f2nas_2024].

Solution 1.9a: Adapt local epochs to an energy-convergence bound. The number of local epochs can be selected in each round from an upper bound that relates loss reduction to client computation and communication costs. This balances additional local work against another cloud-client exchange instead of using one fixed value throughout the search [f2nas_2024]. The cited evaluation reports an energy reduction of nearly 50%, although applying the mechanism requires estimates of client energy and latency.

3.2 Client Heterogeneity

Clients within an FL system can differ in their operational capabilities and learning contexts. This theme covers how these differences affect clients' ability to perform search work and the suitability of architectures for their data, tasks and deployment constraints.

3.2.1 Hardware Heterogeneity

Challenge 2.1: Architecture Search on Heterogeneous Hardware

Differences in client computation and memory make a uniform search workload inefficient or infeasible. A candidate that is inexpensive for one client can exceed another client's memory or extend the round until that client finishes. In weight-sharing search, assigning only hardware-feasible subnets also means that clients train different parts of the supernet, creating unequal exposure across operations and widths [supernet_trade-off_fednas_2025, 4, 40].

Solution 2.1a: Match local search work to client capacity. Supernet-based strategies can assign each client a feasible subspace or sampled subnet and adjust its local depth or optimisation budget to the available computation and memory [apons_2023, divide-and-conquer_fednas_2023, supernet_trade-off_fednas_2025, 4, 40, 44]. The shared search can consequently include weak and strong clients without requiring each of them to execute the complete supernet.

Solution 2.1b: Group clients by training latency. Split-model strategies can profile client computation and communication times and form device groups whose segments have similar completion times. The grouping can be adjusted from observed round latency so that supernet training spends less time waiting for the slowest segment [13].

Solution 2.1c: Tolerate slow or stale clients. A soft-synchronous search can proceed after most selected clients respond and incorporate sufficiently recent late updates using delay compensation [43]. Differentiable search can instead reduce the aggregation weight of an update as its staleness increases [2]. These mechanisms prevent slow clients from blocking the search or giving stale updates full influence.

Solution 2.1d: Account for unequal supernet exposure. When clients train different feasible subnets, the server can aggregate each operation only from clients that updated it and weight those updates by the number of examples that passed through the operation [supernet_trade-off_fednas_2025, 4, 44]. This prevents untrained parameters from being treated as ordinary client updates.

Solution 2.1e: Keep the full search model on the server. The server can retain the complete supernet and send each client only a feasible sampled subnet [rafl_2024, 42]. Clients can also train lightweight knowledge models whose predictions guide server-side supernet training through distillation [41]. These designs keep the largest search structure off constrained devices.

Challenge 2.2: Searching Architectures for Heterogeneous Deployment Hardware

The clients involved in NAS-in-FL may also be the deployment targets. A single architecture sized for the least capable client leaves stronger hardware underused, while one sized for the strongest client may be infeasible elsewhere [4]. The search must therefore produce architectures for several inference-time computation, memory or latency constraints.

Solution 2.2a: Search separate architectures for hardware tiers. Clients with similar inference constraints can be assigned to capability tiers, with one feasible architecture searched for each tier [4, 47]. Shared supernet weights allow these tier-specific architectures to learn from one federated search while stronger tiers retain greater model capacity.

Solution 2.2b: Train a supernet for constraint-specific selection. A weight-sharing supernet can expose subnets of different depths, widths or exits. After federated training, each client selects a subnet that satisfies its hardware limits and performs well on its validation data [18, 41, 50]. This supports finer specialisation than a fixed set of hardware tiers without training an independent model for every client.

Solution 2.2c: Include hardware costs in the search objective. Candidate-based and supernet-based strategies can optimise predictive performance subject to a measured

latency or multiply-accumulate operation budget [47, 20, 44]. Supplying a different hardware profile or budget for each client group makes the selected architecture specific to its deployment hardware.

3.2.2 Client Data Heterogeneity

Challenge 2.3: Differing Client Data Distributions

Differences in class balance and feature distributions can cause clients to favour different operations or rank the same candidates differently [24, 18]. In weight-sharing search, the resulting gradient conflicts destabilise supernet training and make inherited-weight candidate rankings unreliable [41]. Aggregating these signals without accounting for their distributions can select an architecture suited mainly to the dominant client data.

Solution 2.3a: Train the weakest subnets on each data partition. A robust supernet objective can target the candidate with the highest loss on each client partition instead of minimising only average candidate loss. Sampling the smallest, largest and random subnets provides a tractable approximation and reduces interference when different architectures perform poorly on different distributions [18].

Solution 2.3b: Cluster clients by data distribution. Clients with complementary class distributions can be grouped so that each candidate is trained on a cluster whose combined data approximate the global distribution [21]. When different distributions require different architectures, clients can instead be clustered by the similarity of their locally learned architecture variables and train one architecture per cluster [electricity_load_forecasting_fl_darts_2023].

Solution 2.3c: Apply fairness-aware aggregation after search. Once a common architecture has been selected, clients whose local model performs better than the aggregated model can receive greater weight during final model training. This reduces cross-client risk gaps and prevents the trained weights from compounding distributional disparities [mri_generalizable_reconstruction_fednas_2025]. The mechanism changes the weights of the selected architecture rather than the preceding architecture search.

Solution 2.3d: Smooth client weights after architecture selection. During post-search training, each client can maintain an exponential moving average of its model weights rather than relying only on the latest local update. This reduces sensitivity to variation in successive updates. The cited evaluation reports improved reconstruction performance under heterogeneous data [38]. Like Solution 2.3c, the mechanism changes training of the selected architecture rather than the architecture search.

Challenge 2.4: Imbalanced Client Data Volumes

Clients can hold substantially different numbers of examples. Candidate estimates from small local datasets are typically less reliable, while sample-weighted aggregation gives data-rich clients more influence over shared weights and architecture decisions [9]. NAS-in-FL must therefore determine whether the searched architecture should represent the pooled examples or the participating clients.

Solution 2.4a: Choose aggregation weights to match the search objective. Weighting architecture and model updates by local sample count makes the search represent the pooled training examples [9]. Equal client influence can instead be approximated by uniformly averaging client predictions on shared unlabelled data and distilling the result into sampled subnets [41]. The aggregation unit should follow whether performance is evaluated across examples or across clients.

Challenge 2.5: Clients Solve Related but Different Tasks

Client heterogeneity can extend beyond different distributions for one prediction task. Related local tasks may require different label mappings or structural choices, so a shared architecture and one set of weights can produce incompatible predictions even when every client has sufficient resources [11]. Distribution-aware aggregation alone cannot reconcile objectives that require different outputs for the same input.

Solution 2.5a: Factor the architecture into shared and task-specific components. The search space can separate a base component that captures knowledge shared across tasks from a component that clients adapt structurally. A task-conditioned sampling distribution selects the client-specific component, while local fine-tuning adapts it to the corresponding task [11]. This retains collaborative learning but adds a personalisation phase after the shared search.

Challenge 2.6: Client Distributions Change over Time

Cross-client heterogeneity may also vary over time. A fixed architecture selected from historical data can become unsuitable when a client’s latent operating regime changes, particularly in non-stationary time-series applications [2]. Solutions that address only static differences between clients do not allow the active structure to change with the current regime.

Solution 2.6a: Condition architectural choices on the current regime. A lightweight gate can infer a latent regime and activate corresponding operator adapters within a shared supernet. Clients still collaborate on common weights and architecture distributions, while the active subpath changes with the observed regime [2]. The

available evidence is specific to financial time series, so transfer to other forms of temporal drift remains to be established.

Challenge 2.7: Proxy Data Can Misrepresent Private Target Data

Conventional NAS can reduce search cost by using a small proxy dataset, but the resulting architecture may rank differently on the target data. In NAS-in-FL, non-IID client distributions can enlarge this gap and the private target data cannot be pooled to construct a representative central proxy [privacy-preserving_nas_across_fl_iot_2021]. A search space designed for one dataset may create a related transfer problem when applied to different federated data [49].

Solution 2.7a: Search directly on private target data. Clients can update sampled architectural paths on their local target data and aggregate the resulting model and architecture variables without transferring raw examples. Hardware latency can be included in the same objective so that direct search also reflects the deployment platform [privacy-preserving_nas_across_fl_iot_2021]. Server-side search-space preselection, as described in Solution 4.1b, is an alternative when representative server data are available.

3.3 Federated Search-State Coordination

FL distributes the variables, model weights and evidence that guide NAS across clients and rounds. This theme covers mechanisms that combine, preserve or jointly optimise that state so the search follows a coherent trajectory.

3.3.1 Coordinating Distributed Search State

Challenge 3.1: Architecture and Model Parameters Form Coupled Global State

Differentiable NAS jointly optimises model weights and architecture variables. The architecture variables determine which candidate operations contribute to the model, while their gradients depend on the current operation weights. Aggregating only model weights discards local structural decisions, whereas combining architecture variables with unrelated weights separates those decisions from the state that produced them [9, 7, 30].

Solution 3.1a: Aggregate architecture and model variables together. Clients can return both variable sets after local search, after which the server aggregates and broadcasts the resulting architecture variables with their corresponding model weights [9, 7, 30]. This maintains one shared differentiable search state across rounds. Parameter-wise

aggregation requires clients to use the same supernet topology and assign the same meaning to every architecture variable.

Challenge 3.2: Locally Searched Architectures Can Be Structurally Incompatible

Independent local searches can return discrete graphs with different connections, operations or blocks. Their parameters and architecture variables do not occupy a uniform coordinate system, so ordinary parameter or gradient averaging can lose structural information [multi-granular_fednas_2023, 36]. Aggregating only the components shared by several local architectures also discards knowledge learned by their non-overlapping components [f2nas_2024].

Solution 3.2a: Aggregate recurring subgraphs. The server can estimate how often connections and operations occur across local graphs and sample a global graph from the resulting distributions [multi-granular_fednas_2023]. A related strategy directly assembles frequently recurring subgraphs [36]. Both preserve structural information without requiring parameter-wise alignment of complete local architectures.

Solution 3.2b: Transfer knowledge from non-overlapping components. Parameters of compatible blocks can be aggregated with weights based on their similarity, while predictions from heterogeneous blocks can train the server model through distillation [f2nas_2024]. This complements the model-agnostic knowledge transfer in Solutions 2.1e and 2.8b. Exchanging compressed predictions similarly enables collaboration between entirely different local topologies [hetefed_2023]. Distillation avoids direct tensor alignment, but approaches that evaluate predictions on common inputs require a suitable shared dataset.

Challenge 3.3: Search-Policy Knowledge Can Be Lost between Rounds

Controller-based reinforcement-learning strategies accumulate search knowledge in a policy that samples candidate architectures. If client state is reset between federated rounds and only candidate weights are retained, the controller loses what earlier evaluations taught it and restarts its local search from a cold state [29].

Solution 3.3a: Persist and redistribute search-policy state. Clients can upload controller parameters alongside their candidate-weight dictionary. The server retains a selected controller state, aggregates the shared weights and redistributes both for the next round, allowing reinitialised clients to continue from previous search knowledge [29]. The cited framework is conceptual and does not empirically compare rules for selecting or combining client controllers.

3.3.2 Coordinating Interdependent Search Decisions

Challenge 3.4: Architecture Quality Depends on the Federated Training Configuration

An architecture’s apparent quality depends on the configuration under which it is trained and evaluated. Learning rates interact with architectural choices, while the number of local epochs and participating-client fraction change the federated trajectory used to obtain candidate fitness [30, 32, 16]. Tuning these variables only after architecture search can therefore invalidate the comparisons that led to the selected architecture.

Solution 3.4a: Coordinate architecture and hyperparameter optimisation. An alternating procedure can select a training configuration, search an architecture under that configuration and use the result to guide the next hyperparameter decision [30]. Evolutionary search can instead encode the architecture and its training hyperparameters in the same individual and evaluate them together through federated training [32].

Solution 3.4b: Include FL control variables in the search space. Black-box strategies can search the number of local epochs and the fraction of active clients alongside architecture and conventional hyperparameter choices. Evaluating each complete configuration through federated training makes candidate fitness reflect the FL procedure with which the architecture will operate [16].

3.3.3 Coordinating Vertically Partitioned Search

Challenge 3.6: No Party Can Evaluate an End-to-End Candidate Alone

In vertical FL, parties hold different features for the same entities and operate separate parts of a joint model. No feature holder can calculate the complete prediction, loss or architecture gradient because its output must be combined with those of the other parties. The local submodels can also have different topologies, making direct parameter aggregation unsuitable [23].

Solution 3.6a: Coordinate local searches through intermediate representations. Each party can retain its own supernet, weights and architecture variables while participating in a split-model forward and backward pass. Feature-holding parties send intermediate outputs to an aggregating party, which computes the joint loss and returns the corresponding gradients. Each party then updates its local weights and architecture variables without requiring compatible local architectures [23].

Challenge 3.7: Coordinated Vertical Search Is Restricted to Aligned Samples

The joint forward and backward pass requires records that can be matched across all parties. Their aligned intersection may be much smaller than the union of their local datasets, leaving most observations unable to contribute to the supervised architecture-search objective [23].

Solution 3.7a: Pretrain local search state on unaligned data. Before coordinated search, each party can perform self-supervised one-shot NAS on all of its local observations, updating both model weights and architecture variables. These states initialise supervised vertical search on the smaller aligned dataset [23]. Unaligned data therefore shape the initial representations and structural preferences, while the joint phase supplies the cross-party supervised objective.

3.4 Disrupted Architecture Search

Changes in client participation and unstable structural decisions can interrupt architecture search or make its evidence unreliable. This theme covers responses that preserve participation continuity, representative client influence and optimisation stability.

3.4.1 Participation Disruption and Bias

Challenge 1.10: Client Dropout Interrupts Architecture Search

Cross-device clients can become unavailable because of changing network conditions, battery constraints or competing local workloads. A search procedure that requires the same cohort in every round can then stall or lose updates for the candidates or supernet paths assigned to disconnected clients [48].

Solution 1.10a: Design the search for partial participation. The server can require only a random subset of clients to participate in each round and aggregate the search updates returned by that subset. The participating clients may change between rounds, allowing architecture search to continue without requiring every registered client to remain available [48].

Challenge 1.11: Clients Reach Architecture Decisions at Different Times

In personalised differentiable search, clients can optimise architecture variables at different rates because their data distributions and local trajectories differ. A common stopping round can terminate some searches prematurely while allowing others to overfit, for example by increasingly favouring skip connections [24].

Solution 1.11a: Stop architecture updates independently. After a shared warm-up period, each client can stop updating its architecture variables when a local criterion detects increasing preference for skip connections. Other clients continue searching until they meet their own stopping criteria [24].

Challenge 2.8: Bias from Overrepresenting Frequently Available and Well-Connected Participants

Differences in bandwidth and availability cause some clients to participate more often or meet more round deadlines than others [48, 40]. Resource-aware assignment can also make small candidates train mainly on low-capability clients and large candidates on well-connected clients [4]. Candidate quality then becomes entangled with the non-IID data of the clients able to train it, which can bias shared weights and architecture rankings towards frequently available participants [41].

Solution 2.8a: Balance connectivity and data coverage jointly. Candidate-training groups can be constrained by bandwidth while also being adjusted to cover representative class distributions. The literature implements this by rebalancing bandwidth-based groups according to class distribution, or by minimising each cluster’s distance from the population distribution under a completion-time limit [27, 21]. This makes candidate comparisons less dependent on which client links can support them.

Solution 2.8b: Decouple client knowledge from candidate assignment. Client predictions or feature maps on shared inputs can be aggregated into a teaching signal and distilled into different candidate architectures. Sampled supernet paths or personalised models then learn from several clients, including those unable to execute that candidate directly [41, 24]. This transfers knowledge across architecture boundaries without requiring every client to train every candidate.

3.4.2 Structural Instability

Challenge 3.5: Structural Decisions Can Destabilise the Search

Distributed architecture updates can favour different operations at different network depths, and an early preference for parameter-free operations can prevent the architecture from converging [19]. Progressive selection creates a related problem because fixing an operation changes the computation graph and its weight-sharing relationships. The next decision can then be evaluated before the remaining weights have adapted to the preceding structural change [26]. Unstructured subnet sampling can also create jagged aggregate updates and slow supernet convergence [**divide-and-conquer_fednas_2023**].

Solution 3.5a: Regularise layer-wise architecture updates. During local differentiable search, a gradient-alignment term can constrain the relationship between layer

gradients and architecture variables. Incorporating this term into the architecture objective counteracts depth-dependent operation preferences before the client update is aggregated [19].

Solution 3.5b: Separate structural decisions with recovery rounds. A progressive supernet search can first warm up the shared weights and then fix one operation at a time according to its effect on local validation accuracy. Several recovery rounds update the remaining weights after each decision before the next edge is evaluated [26]. Once the relevant edges have been resolved, the architecture is fixed and ordinary training continues.

Solution 3.5c: Vary subnet coverage over the search. A server can assign subnet masks with large pairwise distances during early rounds and progressively reduce those distances as training advances. Early rounds explore separated regions of the search space, while later rounds concentrate updates around better understood regions [**divide-and-conquer_fednas_2023**]. The structured masks also fit together more consistently during aggregation than independently sampled subnets.

3.5 Architecture Search Security

Architecture search exposes structural signals and decisions in addition to conventional model updates. This theme covers integrity, confidentiality and adversarial-robustness risks introduced or addressed during search and deployment.

Challenge 4.1: Poisoning the Architecture Search

Malicious clients can manipulate model weights and architecture-search signals through poisoned data or local optimisation. The attack can therefore pursue a backdoor objective while changing operation probabilities or candidate rankings. In weight-sharing search, compromised weights can also affect every candidate that reuses them. Experiments with one malicious client among eight found substantial degradation in two discrete weight-sharing strategies under fourfold-scaled updates, showing that poisoning can persist in the selected architecture [49].

Solution 4.1a: Use a trained differentiable search state to attenuate poisoned operations. In differentiable supernet-based search, each candidate operation remains part of a weighted mixture while clients update both its weights and selection probability. The cited analysis derives a reduction in an operation’s mixture contribution when its poisoned weights provide an opposing or comparatively small gradient for the main task. This result assumes Lipschitz continuity, convexity, bounded gradients and a unique global optimum [49]. The evaluation supports the mechanism only for the studied backdoor attacks and search spaces. Retraining the selected fixed architecture from

scratch did not preserve the reported robustness, so the evidence concerns the trained differentiable search state rather than intrinsic robustness of the architecture [49].

Solution 4.1b: Restrict the search space before federated search. The server can evaluate modules on server-held data and select a compact set that balances predictive performance, memory and latency before clients begin differentiable search. Constraining model complexity in this way reduced overfitting to noise introduced by backdoor triggers in the cited evaluation [49].

Challenge 4.2: Architecture-Search Artefacts Create Additional Privacy Exposure

NAS-in-FL communicates data-dependent signals that fixed-architecture FL does not require. Differentiable strategies can expose architecture gradients, while other strategies may transmit discrete architecture graphs, personalised choices or scalar rewards [dp-fnas_2020, multi-granular_fednas_2023, 49, 30]. The cited literature treats local sample counts, class proportions and example reconstruction as possible attack targets rather than demonstrating their recovery [multi-granular_fednas_2023]. It also proposes that personalised architecture choices could help identify the source of an update, but does not evaluate such an attack [49]. These data-dependent artefacts therefore constitute a potential privacy exposure whose practical severity depends on the transmitted representation and adversary.

Solution 4.2a: Apply differential privacy to search signals. Differentiable strategies can clip model and architecture gradients and add Gaussian noise before upload, while scalar hyperparameter rewards can be perturbed in the same way [dp-fnas_2020, 30]. In vertically partitioned search, clipping and noise can instead protect the intermediate activations and backward gradients exchanged between parties [23]. The added noise can, however, impair model and search performance [49].

Solution 4.2b: Securely aggregate a uniform shared search. When every client updates the same differentiable supernet, model weights and architecture variables have aligned shapes and can use the same weighted secure-aggregation protocol. Personal architecture selection and adaptation can then occur locally after the shared search, preventing the server from observing each client’s deployed choice [49]. This requires a uniform search representation during collaboration.

Solution 4.2c: Exchange discrete architecture encodings separately from model parameters. The graph aggregation described in Solution 3.2a can upload discrete architecture encodings while handling model-parameter training in a separate stage. It therefore constructs the next global architecture without receiving the corresponding weights or continuous gradients [multi-granular_fednas_2023]. For a seven-layer example, the cited analysis compares a 56-entry derivative matrix with the 50,176 pixels in an image batch and argues that the resulting optimisation has many possible solutions, making precise

pixel reconstruction by that procedure impractical [multi-granular_fednas_2023]. This dimensionality argument does not exclude property or source inference and does not provide a formal privacy guarantee, so the data-dependent encoding may still require differential privacy or encryption.

Challenge 4.3: Adversarial Examples Can Evade the Deployed Model

Poisoning manipulates the search or training process, while an evasion attack perturbs inputs after deployment. An attacker with access to a compromised client can train a replacement model on local data and use NAS to find an architecture whose adversarial examples transfer effectively to the federated target model [fednas_based_attack_2024]. The deployed architecture therefore affects robustness to an attack that does not modify federated updates.

Solution 4.3a: Search jointly for clean performance and adversarial robustness. Candidate architectures can be evaluated through federated adversarial training using both unmodified and adversarial examples. Treating clean predictive performance and adversarial accuracy as separate objectives produces a set of trade-off architectures from which a deployment can be selected [fednas_based_attack_2024]. The cited evidence concerns intrusion detection and specific transfer attacks, so robustness to other domains and adversaries requires separate evaluation.

4 Conclusion

- the amount of possible knobs to turn in NAS-in-FL is astronomical and the literature has only investigated a small subset of them - NAS in FL trails behind NAS and FL - provide some clear direction for future research

5 Discussion

- the further the FL system deviates from centralised NAS, the more challenging it is => conversely, when NAS-in-FL degenerates to centralised NAS in some aspects it becomes simpler - show how some solutions limit compatibility with others and how they are - the search space trade-off - suprisingly many NAS-in-FL methods do X - we expected federation scale to play a more prominent role, but this was barely ever considered by NAS-in-FL methods no large federation scale - similar to NAS research, supernet-based methods have become more popular over time - performance evaluation is basically universally just training a candidate architecture for a couple of epochs, making the main differentiating factors for challenge applicability the NAS search strategy type and secondarily the search space - The FL system characteristics which NAS-in-FL methods apply to tend to be underspecified

5.1 Principal Findings

The findings show that identifying a suitable solution for using NAS in FL requires more than matching a named method to a general challenge. Limited client resources, heterogeneity, architecture-search coordination, and security affect different parts of the search and become relevant under different FL system characteristics. Their effects also overlap. A client's bandwidth can limit the feasible search state, unequal bandwidth can bias which data inform candidate evaluation, and the resulting structural differences can complicate aggregation. The research problem is therefore best understood as a conditional mapping between a system context, the challenge created in that context, and a compatible solution. This complements method-level classifications and archetypes, which explain how complete NAS-in-FL methods are designed but do not by themselves establish which component is suitable for a particular problem [10].

Resource constraints illustrate why this contextual distinction matters. On weak cross-device clients, local supernet training may exceed compute or memory budgets. Single-path training, progressive pruning, lightweight search spaces, and nested weight sharing reduce this local burden [7, 40, 20]. Candidate-based searches face a different cost because every evaluation can invoke several FL rounds. Limiting fine-tuning, predicting performance, evaluating candidates in parallel, or inheriting supernet weights reduces repeated evaluation, while weight inheritance and one-stage optimisation can avoid

a separate retraining stage [45, 41, 39]. A shared supernet can further serve several deployment targets, and split-model searches can use otherwise idle client time [18, 13]. Each solution has different preconditions: a restricted search space requires domain knowledge, parallel evaluation requires enough available clients, and inherited weights provide a cheaper but potentially less faithful ranking than full candidate training.

The communication findings reveal a similar separation between message size, synchronisation frequency, and the communication cost induced by the selected model. Compression and compact architecture encodings address oversized search messages; infrequent or hierarchical aggregation addresses repeated wide-area synchronisation; and communication-aware objectives prevent the search from producing a model that remains expensive in later FL rounds [2, 21, 51]. Compact encodings are applicable only when clients already share the search space and weights. Hierarchical aggregation presupposes suitable intermediate infrastructure. Consequently, a cross-silo system with capable servers but expensive inter-organisational links may need communication solutions without the compute reductions required by a mobile fleet.

Heterogeneity and unreliability show that feasibility and representativeness must be considered. Matching subnets or workloads to client capacity, grouping clients by latency, tolerating stale updates, and keeping the full search model at the server can prevent weak clients from delaying the search [rafl_2024, 4, 13, 43]. However, clients then train different portions or candidates. Exposure-aware aggregation is needed where parameters overlap, while tier-specific architectures, constraint-specific subnet selection, and hardware-aware objectives address heterogeneous deployment budgets [44, 47, 18]. When availability and connectivity correlate with non-IID data, selecting only fast clients can make candidate comparisons reflect client assignment rather than architecture quality. Unequal sample volumes change estimate reliability and client influence on shared architecture decisions [9]. Forming groups from both network conditions and data coverage, or transferring aggregated knowledge across candidate boundaries, preserves evidence [27, 21, 41]. Partial participation addresses dropout, whereas independent stopping criteria address clients whose architecture variables converge at different times [48, 24]. Treating these as separate contexts prevents a generic “asynchronous” solution from obscuring their different causes. Insufficient capacity bounds what a client can execute, whereas unequal capacity changes whose evidence influences the search; reducing workload addresses the former but may in turn intensify the latter.

Architecture-search coordination depends most directly on the representation of search knowledge. Differentiable searches with a common supernet can aggregate model and architecture parameters together, but this requires identical meanings for every aggregated variable [9, 7]. Controller-based searches instead have to persist their learned policy, while searches whose architecture quality depends on FL hy-

perparameters must alternate or jointly optimise these coupled decisions [29, 30, 32]. Regularisation or warm-up and recovery rounds can stabilise structural decisions, depending on whether instability arises in layer-wise gradients or progressive pruning [19, 26]. Vertical FL forms a distinct context because no feature holder can evaluate an end-to-end candidate alone. Exchanging intermediate representations supplies a joint optimisation signal, and self-supervised local pretraining makes unaligned samples useful before coordinated search on the aligned subset [23].

Security completes the mapping by distinguishing integrity from confidentiality. A malicious client can influence persistent architecture decisions, for which suppressing poisoned operations or restricting the search space provides reported protection [49]. An honest-but-curious party presents another problem: architecture gradients, discrete choices, rewards, and intermediate representations remain data-dependent even though raw data stay local. Differential privacy, secure aggregation of a uniform shared search, and low-dimensional discrete encodings protect different observable search signals [dp-fnas_2020, multi-granular_fednas_2023, 30]. Their applicability follows the threat model and search representation, and privacy noise or a restricted representation can weaken the evidence guiding architecture selection.

5.2 Implications for Practice and Research

- clear guidance on existing related solutions and challenges

For practitioners, the findings support a diagnostic sequence. The intended system should first be characterised by FL partitioning, client compute and memory, link capacity, participation, data distributions, trust assumptions, and deployment budgets. The search representation can then be examined to locate the constrained or exposed state. Only after these conditions are explicit can the relevant solutions be combined. For example, capacity-specific subnets may require exposure-aware aggregation, representative client grouping, compact communication, and protection of architecture updates. Selecting one component without its dependencies can transfer cost, bias, or exposure to another part of the search. This makes the map useful before implementation because it reveals which assumptions must hold for a solution to remain operationally feasible.

For research, this synthesis suggests that solution configurations are a more informative unit of comparison than complete NAS-in-FL methods. It provides a vocabulary for distinguishing where a federated condition affects the search process, the evidence used to guide it, or the architecture eventually deployed. It also exposes dependencies between solution components that method-level comparisons can conceal. The mapping therefore supports condition-specific comparison rather than a universal ranking of

solutions. Its main implication is that the applicable context should form part of every challenge–solution claim.

5.3 Limitations

- rely on literature for facts, i.e. described only challenges and solutions from the literature - few NAS-in-FL methods establish an explicit causal link between FL system characteristics and challenges, therefore this needs to be treated as corrolation evidence, not causal

The literature search limits the coverage of the synthesis. It uses an English-language Scopus query and one iteration of backward and forward citation searching. Publications using different terminology, indexed elsewhere, or published after the search may therefore be absent. Excluding surveys and publications without enough methodological detail improves the evidential basis for coding, but it can omit challenges or solutions reported only at a high level. The identified concepts should consequently be understood as a synthesis of the reviewed corpus rather than an exhaustive account of every possible NAS-in-FL design.

The thematic analysis introduces interpretative limitations. The coding captures how authors describe and motivate challenges and solutions, and the applicability of a solution sometimes has to be inferred from the reported FL configuration or experimental setup. Decisions to merge conceptually similar mechanisms and separate superficially similar challenges are grounded in the coded extracts, but alternative researchers could draw theme boundaries differently. Repeated comparison with the complete corpus provides an audit trail for these decisions without eliminating the researcher judgement inherent in familiarisation, coding, and theme refinement [using_thematic_analysis_2006].

Finally, a relationship in the mapping indicates that the literature presents a solution as addressing a challenge under particular conditions. It does not establish the size of that effect or prove that the solution will transfer to another system. The reviewed studies differ in datasets, search spaces, client populations, hardware, resource models, and evaluation budgets. These differences prevent a quantitative comparison across solutions and make publication frequency unsuitable as a measure of effectiveness, severity, or practical prevalence. The findings can therefore guide conditional selection and hypothesis formation, but they cannot guarantee performance in an unstudied FL context.

5.4 Future Research

Future research should test the challenge–solution relationships under controlled and comparable conditions. A shared evaluation framework could vary client compute and memory, bandwidth, data distributions, participation, and deployment budgets while holding the learning task and search space constant. Evaluations should distinguish search cost from the cost of training and deploying the selected architecture. They should also report wall-clock time, transferred bytes, candidate-ranking fidelity, participating-client coverage, final-model quality, and the assumed threat model. These measurements would reveal which system characteristic explains a solution’s benefit and where its reported applicability ceases.

The interactions between solutions require particular attention. Resource-aware workload assignment can keep weak clients involved while changing their exposure to supernet operations; compact or noisy search signals can reduce communication or leakage while weakening architecture evidence; and personalised deployment can require a compatible aggregation mechanism. Factorial experiments could evaluate these components individually and in combination, including configurations that address several challenges simultaneously. Such comparisons would show whether their benefits accumulate, interfere, or shift costs to another part of the search.

Further studies should evaluate NAS in FL under combinations of conditions that occur together in operational systems. Dropout, stale updates, unequal connectivity, and non-IID data can jointly determine which clients influence architecture selection. Security evaluations should likewise consider both the predictive model and the integrity of the structural decision, while stating which search artefacts an adversary can observe or manipulate. Evidence from cross-device, cross-silo, and vertical FL deployments would test whether relationships derived largely from individual experimental settings remain useful at operational scale.

Future reviews can extend the database coverage, repeat citation searching, involve multiple coders, and validate the challenge definitions with practitioners responsible for deployed FL systems. A common reporting scheme for client characteristics, search-time resources, deployment constraints, participation, architecture overlap, and security assumptions would make new proposals easier to compare. With stronger comparative evidence, the present mapping could support a decision aid that recommends solution configurations for an explicitly described FL system.

Abbreviations

List of Figures

List of Tables

Bibliography

- [1] S. S. P. Avval, N. D. Eskue, R. M. Groves, and V. Yaghoubi. "Systematic review on neural architecture search." In: *Artificial Intelligence Review* 58.3 (Jan. 6, 2025), p. 73. ISSN: 1573-7462. DOI: 10.1007/s10462-024-11058-w.
- [2] Z. Chen, H. Zhang, S. Wang, and J. Chen. "FedRegNAS: Regime-Aware Federated Neural Architecture Search for Privacy-Preserving Stock Price Forecasting." In: *Electronics* 14.24 (2025). DOI: 10.3390/electronics14244902.
- [3] J. Corbin and A. Strauss. *Basics of qualitative research*. Core textbook v. 14. SAGE Publications, 2015. ISBN: 9781412997461.
- [4] L. Dudziak, S. Laskaridis, and J. Fernandez-Marques. *FedorAS: Federated architecture search under system heterogeneity*. 2022. arXiv: 2206.11239 [cs.LG].
- [5] Elsevier. *Scopus*. Abstract and citation database. Accessed 2026-01-17. 2026.
- [6] T. Elsken, J. H. Metzen, and F. Hutter. "Neural architecture search: A survey." In: *Journal of Machine Learning Research* 20.55 (2019), pp. 1–21.
- [7] A. Garg, A. K. Saha, and D. Dutta. *Direct federated neural architecture search*. 2020. arXiv: 2010.06223 [cs.LG].
- [8] M. Hartmann, G. Danoy, and P. Bouvry. *Multi-objective methods in Federated Learning: A survey and taxonomy*. 2025. arXiv: 2502.03108 [cs.LG].
- [9] C. He, M. Annavaram, and S. Avestimehr. *Towards Non-I.I.D. and invisible data with FedNAS: Federated deep learning via neural architecture search*. 2021. arXiv: 2004.08546 [cs.LG].
- [10] N. Henze, S. Rank, D. Deng, M. Lindauer, A. Sunyaev, and N. Kannengießer. "Neural Architecture Search in Federated Learning: A Unified Design Concept." In: *TechRxiv* (2026). DOI: 10.36227/techrxiv.176964091.16274890/v1.
- [11] M. Hoang and C. Kingsford. "Personalized Neural Architecture Search for Federated Learning." In: *1st NeurIPS Workshop on New Frontiers in Federated Learning (NFFL 2021)* ().
- [12] X. Huang and J. Gao. *Federated neural architecture search with model-agnostic meta learning*. 2025. arXiv: 2504.06457 [cs.LG].

- [13] C. Huo, J. Jia, T. Deng, M. Dong, Z. Yu, and D. Yuan. "NASFLY: On-device split federated learning with neural architecture search." In: *2024 IEEE international symposium on parallel and distributed processing with applications (ISPA)*. 2024, pp. 2184–2190. doi: 10.1109/ISPA63168.2024.00298.
- [14] D. Jin, N. Kannengießer, S. Rank, and A. Sunyaev. "Collaborative distributed machine learning." In: 57.4 (Dec. 2024). issn: 0360-0300. doi: 10.1145/3704807.
- [15] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings, R. G. L. D'Oliveira, H. Eichner, S. E. Rouayheb, D. Evans, J. Gardner, Z. Garrett, A. Gascón, B. Ghazi, P. B. Gibbons, M. Gruteser, Z. Harchaoui, C. He, L. He, Z. Huo, B. Hutchinson, J. Hsu, M. Jaggi, T. Javidi, G. Joshi, M. Khodak, J. Konečný, A. Korolova, F. Koushanfar, S. Koyejo, T. Lepoint, Y. Liu, P. Mittal, M. Mohri, R. Nock, A. Özgür, R. Pagh, M. Raykova, H. Qi, D. Ramage, R. Raskar, D. Song, W. Song, S. U. Stich, Z. Sun, A. T. Suresh, F. Tramèr, P. Vepakomma, J. Wang, L. Xiong, Z. Xu, Q. Yang, F. X. Yu, H. Yu, and S. Zhao. *Advances and Open Problems in Federated Learning*. 2021. arXiv: 1912.04977 [cs.LG].
- [16] S. Khan, F. Nosheen, S. S. A. Naqvi, H. Jamil, M. Faseeh, M. A. Khan, and D.-H. Kim. "Bilevel Hyperparameter Optimization and Neural Architecture Search for Enhanced Breast Cancer Detection in Smart Hospitals Interconnected With Decentralized Federated Learning Environment." In: *IEEE Access* 12 (2024), pp. 63618–63628. doi: 10.1109/ACCESS.2024.3392572.
- [17] S. Khan, A. Rizwan, A. N. Khan, M. Ali, R. Ahmed, and D. H. Kim. "A multi-perspective revisit to the optimization methods of Neural Architecture Search and Hyper-parameter optimization for non-federated and federated learning environments." In: *Computers and Electrical Engineering* 110 (2023), p. 108867. issn: 0045-7906. doi: <https://doi.org/10.1016/j.compeleceng.2023.108867>.
- [18] A. Khare, A. Agrawal, A. Annavaajjala, P. Behnam, M. Lee, H. Latapie, and A. Tumanov. *SuperFedNAS: Cost-efficient federated neural architecture search for on-device inference*. 2024. arXiv: 2301.10879 [cs.LG].
- [19] J. Li, S. Wang, R. Yang, M. Gong, Z. Hu, N. Zhang, K. Sheng, and Y. Zhou. "Toward Federated Customized Neural Architecture Search for Remote Sensing Scene Classification." In: *IEEE Transactions on Geoscience and Remote Sensing* 63 (2025), pp. 1–12. doi: 10.1109/TGRS.2025.3537085.
- [20] Y. Li, R. Yao, D. Qin, and Y. Wang. "Lightweight federated learning for on-device non-intrusive load monitoring." In: *IEEE Transactions on Smart Grid* 16.2 (2025), pp. 1950–1961. doi: 10.1109/TSG.2024.3482363.

- [21] J. Liu, J. Yan, H. Xu, Z. Wang, J. Huang, and Y. Xu. "Finch: Enhancing federated learning with hierarchical neural architecture search." In: *IEEE Transactions on Mobile Computing* 23.5 (2024), pp. 6012–6026. doi: 10.1109/TMC.2023.3315451.
- [22] S. Liu, X. Wang, and Y. Jin. "Federated Bayesian Optimization for Privacy-Preserving Neural Architecture Search." In: *2023 IEEE Congress on Evolutionary Computation (CEC)*. 2023, pp. 1–8. doi: 10.1109/CEC53210.2023.10254155.
- [23] Y. Liu, X. Liang, J. Luo, Y. He, T. Chen, Q. Yao, and Q. Yang. "Cross-Silo Federated Neural Architecture Search for Heterogeneous and Cooperative Systems." In: *Federated and Transfer Learning*. Ed. by R. Razavi-Far, B. Wang, M. E. Taylor, and Q. Yang. Cham: Springer International Publishing, 2023, pp. 57–86. ISBN: 978-3-031-11748-0. doi: 10.1007/978-3-031-11748-0_4.
- [24] Y. Liu, S. Guo, J. Zhang, Z. Hong, Y. Zhan, and Q. Zhou. "Collaborative Neural Architecture Search for Personalized Federated Learning." In: *IEEE Transactions on Computers* 74.1 (2025), pp. 250–262. doi: 10.1109/TC.2024.3477945.
- [25] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas. "Communication-efficient learning of deep networks from decentralized data." In: *Proceedings of the 20th international conference on artificial intelligence and statistics*. Ed. by S. Aarti and Z. Jerry. Vol. 54. Proceedings of machine learning research. PMLR, Apr. 2017, pp. 1273–1282.
- [26] E. Mushtaq, C. He, J. Ding, and S. Avestimehr. *SPIDER: Searching personalized neural architecture for federated learning*. 2021. arXiv: 2112.13939 [cs.LG].
- [27] G. Öcal and A. Özgövde. "Network-aware federated neural architecture search." In: *Future Generation Computer Systems* 162 (2025), p. 107475. ISSN: 0167-739X. doi: <https://doi.org/10.1016/j.future.2024.07.053>.
- [28] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, L. Shamseer, J. M. Tetzlaff, E. A. Akl, S. E. Brennan, R. Chou, J. Glanville, J. M. Grimshaw, A. Hróbjartsson, M. M. Lalu, T. Li, E. W. Loder, E. Mayo-Wilson, S. McDonald, L. A. McGuinness, L. A. Stewart, J. Thomas, A. C. Tricco, V. A. Welch, P. Whiting, and D. Moher. "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews." In: *BMJ* 372 (2021). doi: 10.1136/bmj.n71. eprint: <https://www.bmj.com/content/372/bmj.n71.full.pdf>.
- [29] T. Ramírez-Gordillo, H. Mora, F. A. Pujol, and A. Maciá-Lillo. "Federated Neural Architecture Search for Efficient and Privacy-Preserving Model Training." In: *2024 IEEE International Conference on Smart Internet of Things (SmartIoT)*. 2024, pp. 129–136. doi: 10.1109/SmartIoT62235.2024.00028.

- [30] J. Seng, P. Prasad, M. Mundt, D. S. Dhimi, and K. Kersting. *FEATHERS: Federated Architecture and Hyperparameter Search*. 2023. arXiv: 2206.12342 [cs.LG].
- [31] J.-M. Shao, G.-Q. Zeng, K.-D. Lu, G.-G. Geng, and J. Weng. "Automated federated learning for intrusion detection of industrial control systems based on evolutionary neural architecture search." In: *Computers & Security* 143 (2024), p. 103910. DOI: 10.1016/j.cose.2024.103910.
- [32] R. E. Shawi. "AgingFedNAS: Aging evolution federated deep learning for architecture and hyperparameter search." In: *Advances in knowledge discovery and data mining*. Ed. by X. Wu, M. Spiliopoulou, C. Wang, V. Kumar, L. Cao, Y. Wu, Y. Yao, and Z. Wu. Singapore: Springer Nature Singapore, 2025, pp. 107–119. ISBN: 978-981-96-8173-0.
- [33] J. Su, C. Tang, and Z. Liu. "A Prediction Optimization Method with Federated Learning and Neural Architecture Search for Distributed Renewable Energy Sources." In: *Distributed Generation & Alternative Energy Journal* 40.2 (2025), pp. 213–238. DOI: 10.13052/dgaej2156-3306.4021.
- [34] A. Z. Tan, H. Yu, L. Cui, and Q. Yang. "Towards personalized federated learning." In: *IEEE Transactions on Neural Networks and Learning Systems* 34.12 (2023), pp. 9587–9603. DOI: 10.1109/TNNLS.2022.3160699.
- [35] C. Wang, B. Chen, G. Li, and H. Wang. "Automated Graph Neural Network Search under Federated Learning Framework." In: *IEEE Transactions on Knowledge and Data Engineering* 35.10 (2023), pp. 9959–9972. DOI: 10.1109/TKDE.2023.3250264.
- [36] X. Wei, G. Chen, C. Yang, H. Zhao, C. Wang, and H. Yue. "EFNAS: Efficient federated neural architecture search across AIoT devices." In: *2024 international joint conference on neural networks (IJCNN)*. 2024, pp. 1–8. DOI: 10.1109/IJCNN60899.2024.10650653.
- [37] C. White, M. Safari, R. Sukthankar, B. Ru, T. Elsken, A. Zela, D. Dey, and F. Hutter. *Neural architecture search: Insights from 1000 papers*. 2023. arXiv: 2301.08727 [cs.LG].
- [38] R. Wu, C. Li, J. Zou, and S. Wang. "FedAutoMRI: Federated neural architecture search for MR image reconstruction." In: *Medical Image Computing and Computer Assisted Intervention – MICCAI 2023 Workshops*. Berlin, Heidelberg: Springer, 2023, pp. 347–356. DOI: 10.1007/978-3-031-47401-9_33.
- [39] Y. Wu, H. Fan, W. Ying, Z. Zhou, Q. Zheng, and J. Zhang. "Federated Neural Architecture Search with Hierarchical Progressive Acceleration for Medical Image Segmentation." In: *Advances in Swarm Intelligence*. Ed. by Y. Tan and Y. Shi. Singapore: Springer Nature Singapore, 2024, pp. 112–123. ISBN: 978-981-97-7184-4.

- [40] J. Yan, J. Liu, H. Xu, Z. Wang, and C. Qiao. "Peaches: Personalized federated learning with neural architecture search in edge computing." In: *IEEE Transactions on Mobile Computing* 23.11 (2024), pp. 10296–10312. doi: 10.1109/TMC.2024.3373506.
- [41] A. Yang and Y. Liu. "Heterogeneity-aware personalized federated neural architecture search." In: *Entropy* 27.7 (2025). issn: 1099-4300. doi: 10.3390/e27070759.
- [42] D. Yao and B. Li. "PerFedRLNAS: One-for-all personalized federated neural architecture search." In: *Proceedings of the AAAI Conference on Artificial Intelligence* 38.15 (Mar. 2024), pp. 16398–16406. doi: 10.1609/aaai.v38i15.29576.
- [43] D. Yao, L. Wang, J. Xu, L. Xiang, S. Shao, Y. Chen, and Y. Tong. "Federated model search via reinforcement learning." In: *2021 IEEE 41st international conference on distributed computing systems (ICDCS)*. 2021, pp. 830–840. doi: 10.1109/ICDCS51616.2021.00084.
- [44] W. Ying, C. Wang, Y. Wu, X. Luo, Z. Wen, and H. Zhang. "FGFL: Fine-grained federated learning based on neural architecture search for heterogeneous clients." In: *Advances in Swarm Intelligence*. Singapore: Springer Nature Singapore, 2024, pp. 99–111.
- [45] J. Yuan, M. Xu, Y. Zhao, K. Bian, G. Huang, X. Liu, and S. Wang. *Federated neural architecture search*. 2022. arXiv: 2002.06352 [cs.LG].
- [46] J. Yuan, M. Xu, Y. Zhao, K. Bian, G. Huang, X. Liu, and S. Wang. "Resource-Aware Federated Neural Architecture Search over Heterogeneous Mobile Devices." In: *IEEE Transactions on Big Data* (2022). doi: 10.1109/TBDATA.2022.3227403.
- [47] C. Zhang, X. Yuan, Q. Zhang, G. Zhu, L. Cheng, and N. Zhang. "Toward tailored models on private AIoT devices: Federated direct neural architecture search." In: *IEEE Internet of Things Journal* 9.18 (2022), pp. 17309–17322. doi: 10.1109/JIOT.2022.3154605.
- [48] F. Zhang, J. Ge, C. Wong, S. Zhang, C. Li, and B. Luo. "Optimizing Federated Edge Learning on Non-IID Data via Neural Architecture Search." In: *2021 IEEE Global Communications Conference (GLOBECOM)*. 2021, pp. 1–6. doi: 10.1109/GLOBECOM46510.2021.9685909.
- [49] F. Zhang, M. Li, J. Ge, F. Tang, S. Zhang, J. Wu, and B. Luo. "Privacy-Preserving Federated Neural Architecture Search with Enhanced Robustness for Edge Computing." In: *IEEE Transactions on Mobile Computing* 24.3 (2025), pp. 2234–2252. doi: 10.1109/TMC.2024.3490835.

- [50] Z. Zhang, Z. Liu, Y. Zhao, C. Qiu, C. Zhang, and X. Wang. "ENASFL: A federated neural architecture search scheme for heterogeneous deep models in distributed edge computing systems." In: *IEEE Transactions on Network Science and Engineering* 11.3 (2024), pp. 2610–2620. DOI: 10.1109/TNSE.2023.3344850.
- [51] H. Zhu and Y. Jin. "Multi-Objective Evolutionary Federated Learning." In: *IEEE Transactions on Neural Networks and Learning Systems* 31.4 (2020), pp. 1310–1322. DOI: 10.1109/TNNLS.2019.2919699.
- [52] H. Zhu and Y. Jin. *Real-time federated evolutionary neural architecture search*. 2020. arXiv: 2003.02793 [cs.LG].
- [53] H. Zhu, H. Zhang, and Y. Jin. "From federated learning to federated neural architecture search: a survey." In: *Complex and Intelligent Systems* 7.2 (Apr. 2021), pp. 639–657. ISSN: 2198-6053. DOI: 10.1007/s40747-020-00247-z.
- [54] B. Zoph and Q. V. Le. *Neural architecture search with reinforcement learning*. 2017. arXiv: 1611.01578 [cs.LG].